```
"use client";

import { useEffect, useState, useCallback } from "react";
import { PoolConfig, PlayerStanding } from "@/lib/types";
import { computeLeaderboard, formatScore, scoreColorClass } from "@/lib/pool";
import Link from "next/link";

function ScoreBadge({ score }: { score: number | null }) {
  return (
    <span className={`font-mono text-sm ${scoreColorClass(score)}`}>
      {formatScore(score)}
    </span>
  );
}

function RoundDots({ golfer }: { golfer: PoolConfig["golfers"][0] }) {
  const rounds = [golfer.r1, golfer.r2, golfer.r3, golfer.r4];
  return (
    <div className="flex gap-1">
      {rounds.map((r, i) => (
        <span
          key={i}
          className={`w-2 h-2 rounded-full ${r !== null ? "bg-masters-green" : "b
          title={`R${i + 1}: ${formatScore(r)}`}
        />
      ))}
    </div>
  );
}

function StandingRow({ standing, expanded, onToggle }: {
  standing: PlayerStanding;
  expanded: boolean;
  onToggle: () => void;
}) {
  const isLeader = standing.rank === 1;
  const golfersPerRow = 4;

  return (
    <>
      <tr
        className={`cursor-pointer hover:bg-masters-cream transition-colors borde
          ${isLeader ? "bg-amber-50" : ""}`}
        onClick={onToggle}
```

```
>
  {/* Rank */}
  <td className="px-4 py-4 w-12">
    {isLeader ? (
      <span className="inline-flex items-center justify-center w-7 h-7 roun
    ) : (
      <span className="text-gray-500 font-mono text-sm pl-1">{standing.rank
    )}
  </td>

  {/* Name */}
  <td className="px-2 py-4">
    <div className="flex items-center gap-2">
      <span className="font-semibold text-gray-900">{standing.player.name}<
      {isLeader && <span className="text-[10px] bg-masters-gold text-white
    </div>
    <div className="text-xs text-gray-400 mt-0.5">
      {standing.golfers.filter(g => g.counted).length} golfers counted
    </div>
  </td>

  {/* Score */}
  <td className="px-4 py-4 text-right">
    <span className={`text-lg font-mono font-bold ${scoreColorClass(standin
      {formatScore(standing.totalScore)}
    </span>
  </td>

  {/* Prize */}
  <td className="px-4 py-4 text-right text-sm text-gray-500 hidden sm:table
    {standing.prize > 0 ? (
      <span className="text-masters-gold font-semibold">${standing.prize}</
    ) : "—"}
  </td>

  {/* Expand */}
  <td className="px-3 py-4 text-gray-400 w-8">
    <svg
      className={`w-4 h-4 transition-transform ${expanded ? "rotate-90" : "
      fill="none" viewBox="0 0 24 24" stroke="currentColor"
    >
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d=
    </svg>
  </td>
</tr>

{/* Expanded golfer detail */}
```

```
      {expanded && (
        <tr className="bg-masters-cream border-b border-gray-200">
          <td colSpan={5} className="px-6 py-4">
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-2">
              {standing.golfers.map((gs) => (
                <div
                  key={gs.golfer.id}
                  className={`flex items-center justify-between rounded px-3 py-2
                    ${gs.counted ? "bg-white" : "bg-gray-100 opacity-60"}`}
                >
                  <div className="flex items-center gap-2 min-w-0">
                    <RoundDots golfer={gs.golfer} />
                    <span className="truncate font-medium text-gray-800">{gs.golf
                    {gs.golfer.madeCut === false && (
                      <span className="text-[10px] bg-red-100 text-red-600 px-1.5
                    )}
                    {!gs.counted && (
                      <span className="text-[10px] bg-gray-200 text-gray-500 px-1
                    )}
                  </div>
                  <div className="flex items-center gap-3 flex-shrink-0 ml-3">
                    <div className="flex gap-1 text-xs text-gray-400 font-mono hi
                      {[gs.golfer.r1,gs.golfer.r2,gs.golfer.r3,gs.golfer.r4].map(
                        <span key={i} className={r !== null ? scoreColorClass(r)
                          {r !== null ? formatScore(r) : "·"}
                        </span>
                      ))}
                    </div>
                    {gs.penaltyScore > 0 && (
                      <span className="text-xs text-red-500 font-mono">+{gs.penal
                    )}
                    <ScoreBadge score={gs.totalScore} />
                  </div>
                </div>
              ))}
            </div>
          </td>
        </tr>
      )}
    </>
  );
}

export default function LeaderboardPage() {
  const [config, setConfig] = useState<PoolConfig | null>(null);
  const [standings, setStandings] = useState<PlayerStanding[]>([]);
  const [expandedId, setExpandedId] = useState<string | null>(null);
```

```
const [loading, setLoading] = useState(true);
const [lastUpdated, setLastUpdated] = useState<Date | null>(null);

const fetchPool = useCallback(async () => {
  try {
    const res = await fetch("/api/pool");
    const data = await res.json();
    if (data) {
      setConfig(data);
      setStandings(computeLeaderboard(data));
      setLastUpdated(new Date());
    }
  } finally {
    setLoading(false);
  }
}, []);

useEffect(() => {
  fetchPool();
  const interval = setInterval(fetchPool, 30_000);
  return () => clearInterval(interval);
}, [fetchPool]);

if (loading) {
  return (
    <div className="flex items-center justify-center py-24">
      <div className="text-center">
        <div className="w-12 h-12 border-4 border-masters-green border-t-transp
        <p className="text-gray-500 font-serif italic">Loading standings…</p>
      </div>
    </div>
  );
}

if (!config || !config.setupComplete) {
  return (
    <div className="text-center py-20">
      <div className="text-6xl mb-6">⛳</div>
      <h1 className="font-serif text-3xl text-masters-green mb-3">No Pool Confi
      <p className="text-gray-500 mb-8">The commissioner hasn&apos;t set up the
      <Link href="/setup" className="btn-green inline-block">Go to Setup</Link>
    </div>
  );
}

const totalPurse = config.players.length * config.buyIn;
const roundsWithData = [1,2,3,4].filter(r =>
```

```
  config.golfers.some(g => g[`r${r}` as keyof typeof g] !== null)
);
const currentRound = roundsWithData.length > 0 ? Math.max(...roundsWithData) :

return (
  <div>
    {/* Header */}
    <div className="mb-8">
      <h1 className="font-serif text-4xl font-bold text-masters-green mb-1">
        {config.poolName || "Masters Pool 2026"}
      </h1>
      <div className="flex flex-wrap gap-4 mt-3 text-sm text-gray-600">
        <span className="flex items-center gap-1.5">
          <span className="w-2 h-2 rounded-full bg-masters-gold inline-block" />
          Total purse: <strong className="text-gray-900">${totalPurse}</strong>
        </span>
        <span className="flex items-center gap-1.5">
          <span className="w-2 h-2 rounded-full bg-masters-green inline-block"
          {config.players.length} players · {config.golfers.length} golfers
        </span>
        {currentRound > 0 && (
          <span className="flex items-center gap-1.5">
            <span className="w-2 h-2 rounded-full bg-amber-400 inline-block" />
            Round {currentRound} scores entered
          </span>
        )}
        {lastUpdated && (
          <span className="text-gray-400 ml-auto">
            Updated {lastUpdated.toLocaleTimeString()}
          </span>
        )}
      </div>
    </div>

    {/* Settings summary */}
    <div className="flex flex-wrap gap-2 mb-6 text-xs">
      {[
        config.settings.draftType === "snake" ? "🐍 Snake Draft" : "🎲 Random I
        config.settings.scoringType === "all"
          ? "🏌 All Golfers Count"
          : `⭐ Best ${config.settings.bestN} Count`,
        config.settings.missedCutRule === "penalty"
          ? `✂️ MC = +${config.settings.missedCutPenalty}/round`
          : config.settings.missedCutRule === "zero"
          ? "✂️ MC = No Penalty"
          : "✂️ MC = Worst Score",
        config.settings.purseType === "winner-take-all"
```

```
            ? "🏆 Winner Take All"
            : config.settings.purseType === "70-30"
            ? "🥇🥈 70 / 30 Split"
            : config.settings.purseType === "60-30-10"
            ? "🥇🥈🥉 60 / 30 / 10 Split"
            : "🏆 Custom Purse",
        ].map((label) => (
          <span key={label} className="bg-masters-green/10 text-masters-green px-
            {label}
          </span>
        ))}
      </div>

      {/* Standings table */}
      <div className="card overflow-hidden">
        <table className="w-full">
          <thead>
            <tr className="bg-masters-green text-white text-xs uppercase tracking
              <th className="px-4 py-3 text-left w-12">#</th>
              <th className="px-2 py-3 text-left">Player</th>
              <th className="px-4 py-3 text-right">Total</th>
              <th className="px-4 py-3 text-right hidden sm:table-cell">Prize</th
              <th className="w-8 px-3 py-3" />
            </tr>
          </thead>
          <tbody>
            {standings.map((standing) => (
              <StandingRow
                key={standing.player.id}
                standing={standing}
                expanded={expandedId === standing.player.id}
                onToggle={() =>
                  setExpandedId(expandedId === standing.player.id ? null : standi
                }
              />
            ))}
          </tbody>
        </table>
      </div>

      {/* No scores yet */}
      {currentRound === 0 && (
        <p className="text-center text-gray-400 mt-6 italic font-serif">
          Scores will appear here once the commissioner enters Round 1 data.
        </p>
      )}
    </div>
```

```
    );
  }
```